Lab 2: Claude Code Team Workspace

Alignment: Domain 3 - Claude Code Configuration & Workflows Estimated time: 75 minutes Mode: Local simulation; no API key required

Objective

Create an auditable instruction hierarchy with path-specific rules, a reusable skill, an MCP server declaration, and a plan-versus-execute decision gate.

Scenario

A software team needs shared Claude Code behaviour without forcing Python rules onto documentation work or embedding fragile procedures in one long prompt.

Prerequisites

- Python 3.10 or newer.
- A terminal opened in this lab folder.
- No production account, network access, or secret is required.

Architecture before execution

Read the source and identify the control boundary, the evidence artifact, and the terminal condition before running it. All side effects are confined to this lab's evidence/ folder.

Procedure

1. Review the project-level CLAUDE.md **and identify stable rules that apply to every task.**``bashsed -n '1,200p' workspace/CLAUDE.md`### 2. Inspect the Python-scoped rule and explain when it should activate.`bashsed -n '1,200p' workspace/.claude/rules/python.md`### 3. Inspect the reusable review skill and separate its inputs, workflow, outputs, and checks.`bashsed -n '1,240p' workspace/.claude/skills/review/SKILL.md`### 4. Validate the local MCP declaration and confirm no secret value is embedded.`bashpython3 -m json.tool workspace/.mcp.json`### 5. Run the verifier and save its output.`bashmkdir -p evidence && python3 -B verify.py | tee evidence/verification.txt``

Acceptance checks

[ ] Project instructions contain stable policies only.

[ ] Python rules declare a path scope.

[ ] The skill includes verification and a bounded output contract.

[ ] The MCP config uses an environment-variable placeholder.

Reflection

Which instruction belongs at project scope, and which would cause unnecessary context or contradictions if placed there?

Evidence and completion

Save terminal output in evidence/verification.txt. The lab is complete when the verifier prints PASS, every acceptance check is satisfied, and your reflection identifies one design trade-off.

Troubleshooting

- Run commands from this lab folder so relative paths resolve correctly.
- Use python3 -B to prevent learner machines from creating __pycache__ artifacts.
- If a check fails, read the named failed assertion, inspect the referenced file, and rerun only after correcting the underlying design.

Clean-up

Remove only files you created outside this folder. Never store API keys, access tokens, customer data, or production logs in lab evidence.